

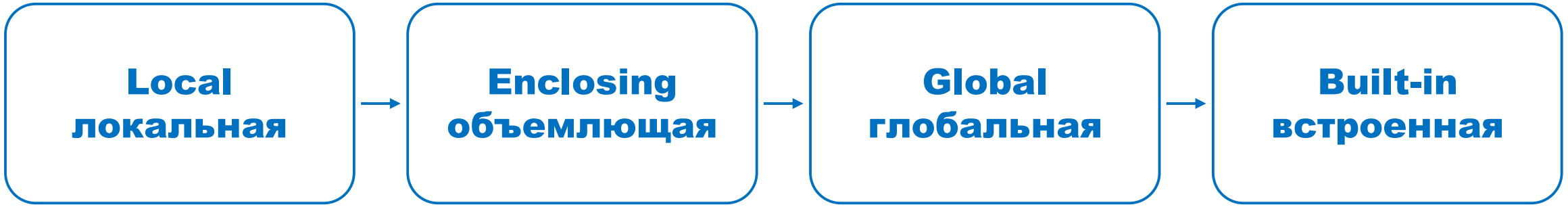
Замыкания и Декораторы

Докладчик: Евграфов Михаил



UnboundLocalError

Правило LEGB

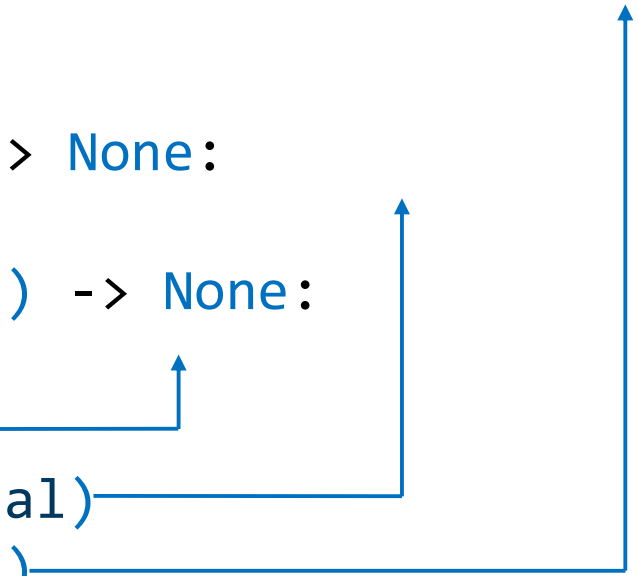


Поиск имен

```
var_global = 42

def nesting_function() -> None:
    var_nonlocal = 142
    def nested_function() -> None:
        var_local = 242
        print(var_local)
        print(var_nonlocal)
        print(var_global)
    nested_function()

nesting_function()
# 242
# 142
# 42
```



Присваивание и локальное имя

```
counter = 0

def reset_counter() -> None:
    counter = 100


reset_counter()
print(counter)
# 0
```

Классификация имен

локальные

параметры
имена с присваиванием в теле
=, +=, for, import, def, with

внешние

имена только читаются
ищутся во внешних областях

Чтение vs присваивание

чтение имени	присваивание
поиск: L, затем E, G, B побеждает ближайшее не найдено: NameError	имя всегда локальное создается в текущей функции

co_varnames и co_names

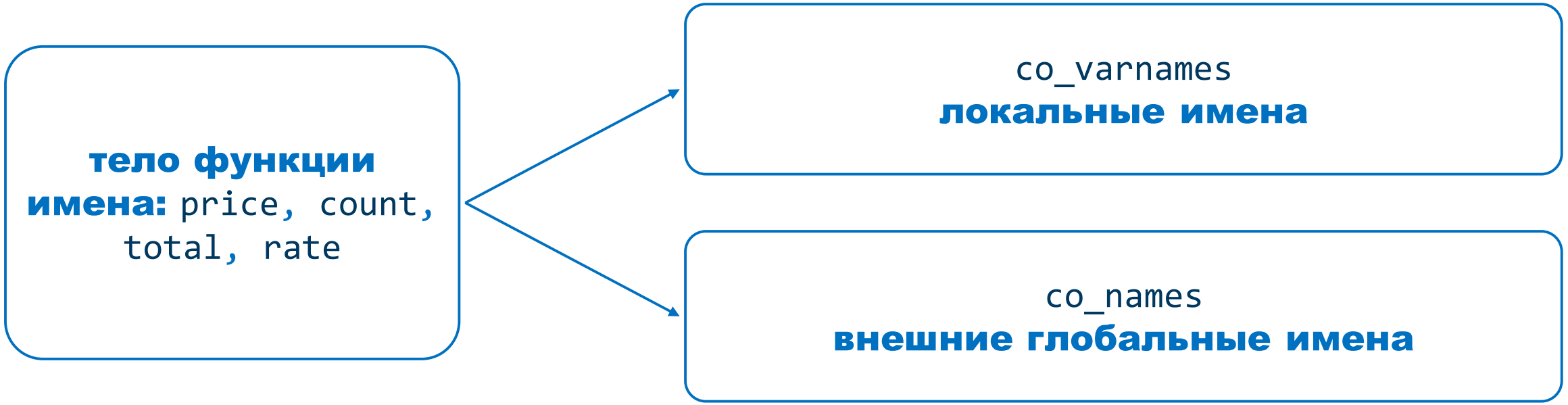
```
rate = 2

def calculate(price: float, count: int) -> float:
    total = price * count
    return total * rate

print(calculate.__code__.co_varnames)
# ('price', 'count', 'total')
print(calculate.__code__.co_names)
# ('rate',)
```

Классификация имен

выполняется при компиляции функции, до ее запуска



UnboundLocalError

```
value = 10

def show_value() -> None:
    print(value)
    value = 20

show_value()
# UnboundLocalError: ...
```

Таблица имен show_value

```
value = 10

def show_value() -> None:
    print(value)
    value = 20

print(show_value.__code__.co_names)
# ('print',)
print(show_value.__code__.co_varnames)
# ('value',)
```

Три типичных случая

чтение до присваивания

составное присваивание

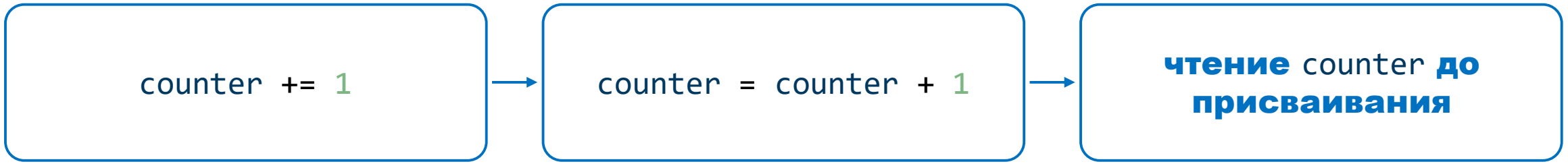
частичное определение

Составное присваивание

```
counter = 0

def increment() -> None:
    counter += 1

increment()
# UnboundLocalError
```



Модификация и +=

```
history = []

def add_with_method(item: str) -> None:
    history.append(item)

def add_with_operator(item: str) -> None:
    history += [item]

add_with_method("first")
print(history)
# ['first']
add_with_operator("second")
# UnboundLocalError: ...
```

Частичное определение

```
def get_label(temperature: int) -> str:
    if temperature > 25:
        label = "hot"

    return label

print(get_label(30))
# hot
print(get_label(10))
# UnboundLocalError: ...
```

Частичное определение

```
def get_last_item(items: list[int]) -> int:
    for item in items:
        pass

    return item

print(get_last_item([1, 2, 3]))
# 3
print(get_last_item([]))
# UnboundLocalError: ...
```

Как избежать ошибки

параметры и return

инициализация на всех путях

global и nonlocal

Параметры и return

```
def increment(counter: int) -> int:
    return counter + 1

counter = 0
counter = increment(counter)

print(counter)
# 1
```

Инициализация до условия

```
def get_label(temperature: int) -> str:
    label = "normal"

    if temperature > 25:
        label = "hot"

    return label

print(get_label(30))
# hot
print(get_label(10))
# normal
```



global и nonlocal

Инструкция global

```
counter = 0

def increment() -> None:
    global counter
    counter += 1

increment()
increment()
print(counter)
# 2
```

global и таблица имен

```
counter = 0

def increment() -> None:
    global counter
    counter += 1

print(increment.__code__.co_varnames)
# ()
print(increment.__code__.co_names)
# ('counter',)
```

Создание имени через global

```
def create_setting() -> None:
    global debug_mode
    debug_mode = True

create_setting()
print(debug_mode)
# True
```

global после использования

```
def broken() -> None:
    print(counter)
    global counter

# SyntaxError: ...
```

Когда global не нужен

```
history = []

def add_to_history(item: str) -> None:
    history.append(item)

add_to_history("first")
print(history)  # ['first']
```

Минусы использования global

результат зависит от состояния модуля

вызов меняет состояние остального кода

сложнее тестировать и отлаживать

Enclosing и затенение

```
def outer() -> None:
    message = "outer"

    def inner() -> None:
        message = "inner"

    inner()
    print(message)

outer()
# outer
```

Enclosing и +=

```
def outer() -> None:
    counter = 0

    def inner() -> None:
        counter += 1

    inner()

outer()
# UnboundLocalError: ...
```

Инструкция nonlocal

```
def outer() -> None:
    counter = 0

    def inner() -> None:
        nonlocal counter
        counter += 1

    inner()
    inner()
    print(counter)

outer()
# 2
```

nonlocal и таблица имен

```
def outer() -> None:
    counter = 0

    def inner() -> None:
        nonlocal counter
        counter += 1

    print(inner.__code__.co_varnames)
    print(inner.__code__.co_freevars)

outer()
# () - локальных имен нет
# ('counter',) - свободная переменная
```

Создание имени через nonlocal

```
counter = 0

def outer() -> None:
    def inner() -> None:
        nonlocal counter
        counter += 1

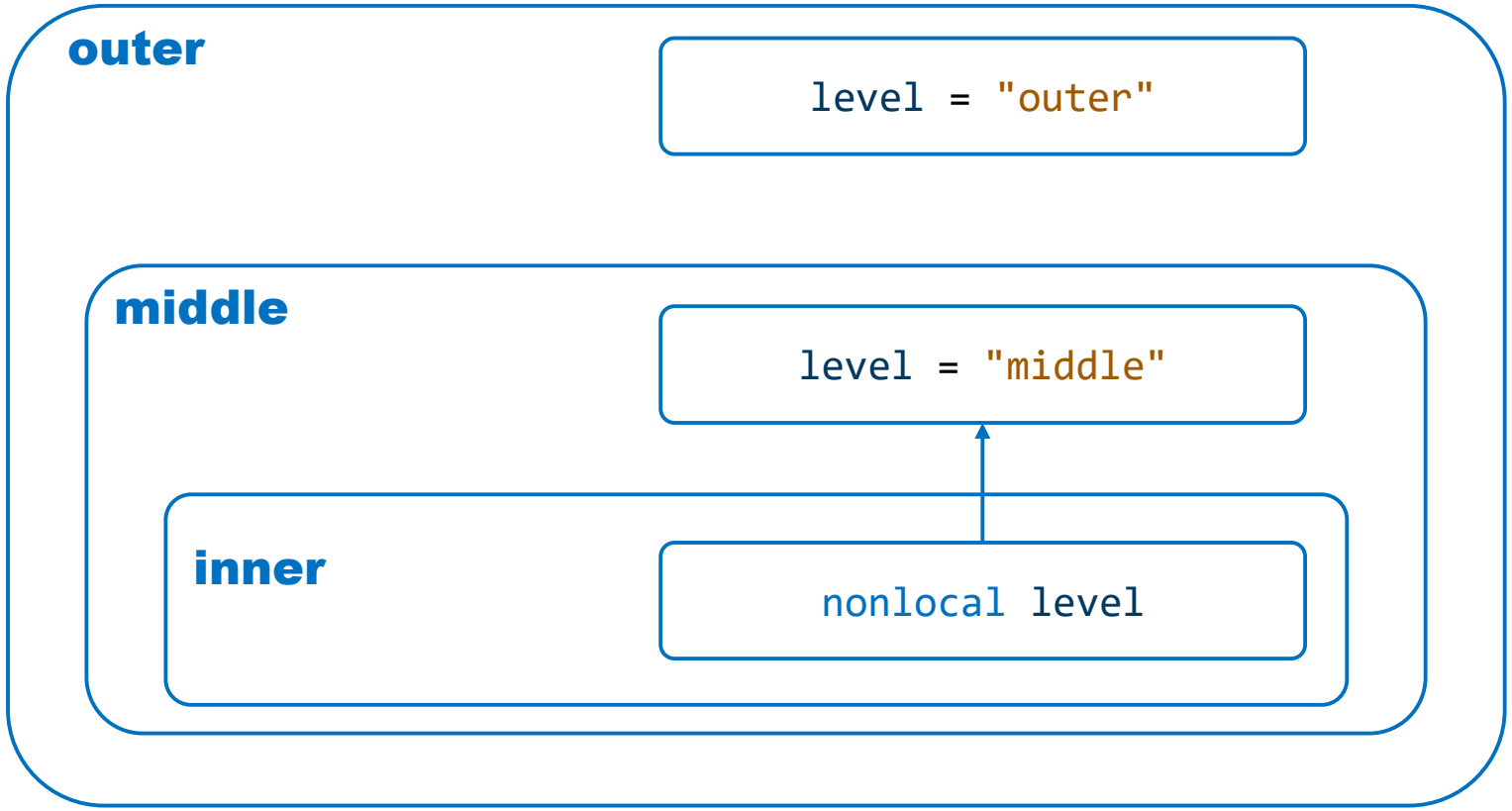
# SyntaxError: ...
```

Поиск имен

```
def outer() -> None:
    level = "outer"
    def middle() -> None:
        level = "middle"
        def inner() -> None:
            nonlocal level
            level = "changed"
        inner()
        print(level)
    middle()
    print(level)

outer()
# changed
# outer
```

Поиск имен



не затронута

изменяется

Когда nonlocal не нужен

```
def outer() -> None:
    items = []

    def inner() -> None:
        items.append(1)

    inner()
    print(items)

outer()
# [1]
```

global и nonlocal

	global	nonlocal
привязывает имя к	**пространству имен модуля**	**ближайшей объемлющей функции**
имя должно существовать	**нет**	**да, иначе SyntaxError**
где применяется	**в любой функции**	**только во вложенной**
в символьной таблице	`co_names`	`co_freevars`



Замыкания

Парадокс

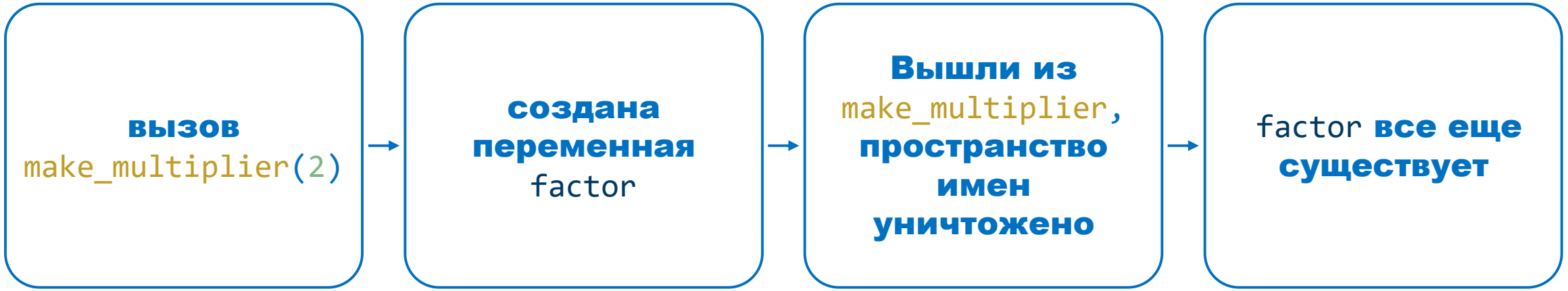
```
from typing import Callable

def make_multiplier(factor: int) -> Callable[[int], int]:
    def multiply(number: int) -> int:
        return number * factor

    return multiply

double = make_multiplier(2)
triple = make_multiplier(3)
print(double(10))  # 20
print(triple(10))  # 30
```

Парадокс



переменная живет дольше вызова внешней функции !!!

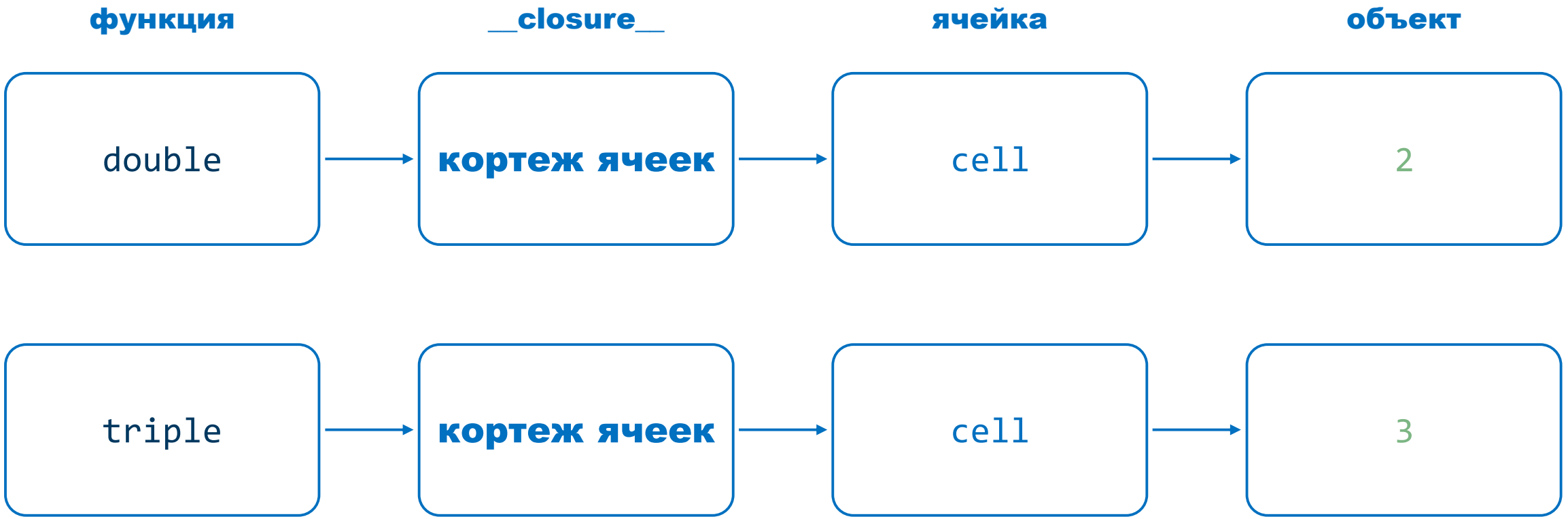
Ячейка

```
double = make_multiplier(2)
triple = make_multiplier(3)

print(double.__code__.co_freevars)
# ('factor',)
print(double.__closure__)
# (<cell at 0x...: int object at 0x...>,)

print(double.__closure__[0].cell_contents)  # 2
print(triple.__closure__[0].cell_contents)  # 3
```

Устройство замыкания



каждый вызов фабрики создает новую ячейку

Запись в ячейки

```
from typing import Callable

def make_printer() -> Callable[[], None]:
    message = "first"

    def print_message() -> None:
        print(message)

    message = "second"
    return print_message

printer = make_printer()
printer()  # second
```

Ошибки работы с ячейками

```
from typing import Callable

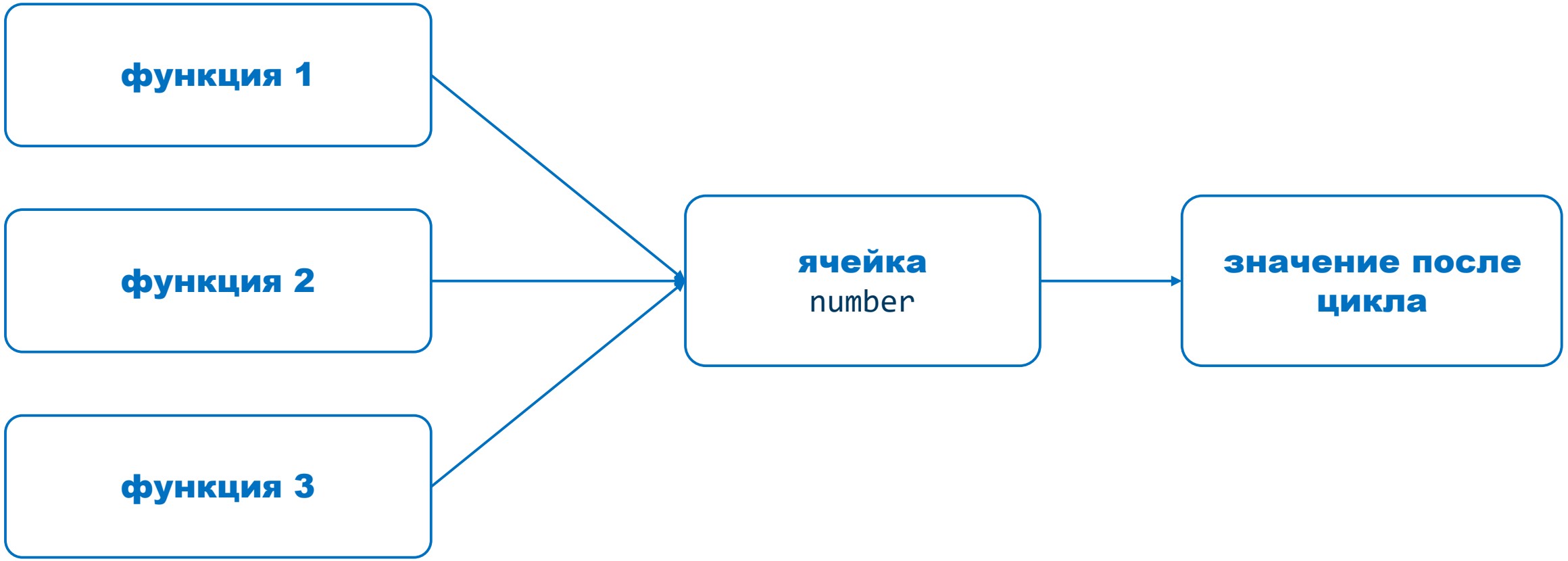
def make_functions() -> list[Callable[[], int]]:
    functions = []

    for number in range(3):
        def get_number() -> int:
            return number
        functions.append(get_number)

    return functions

for function in make_functions():
    print(function())
# 2, 2, 2 - не 0, 1, 2
```

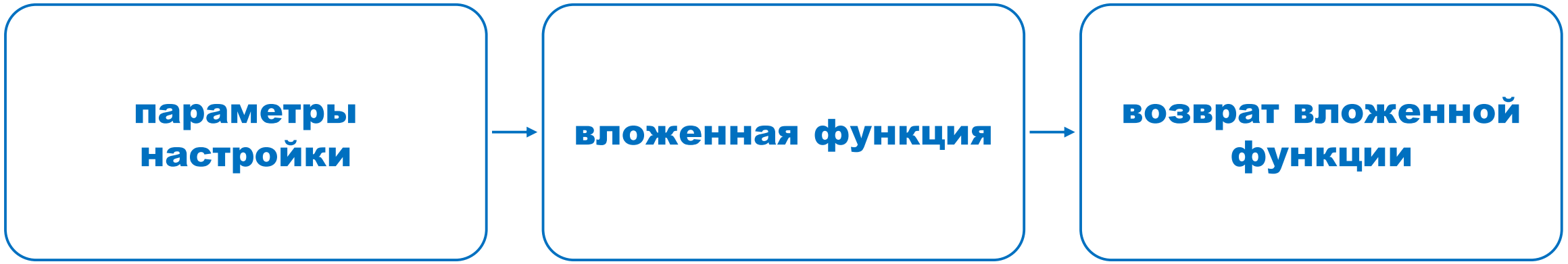
Одна ячейка на всех





Фабрики функций

Шаблон фабрики



Пример фабрики

```
from typing import Callable

# 1. параметры настройки
def make_power(exponent: int) -> Callable[[float], float]:
    # 2. вложенная функция
    def power(number: float) -> float:
        return number ** exponent

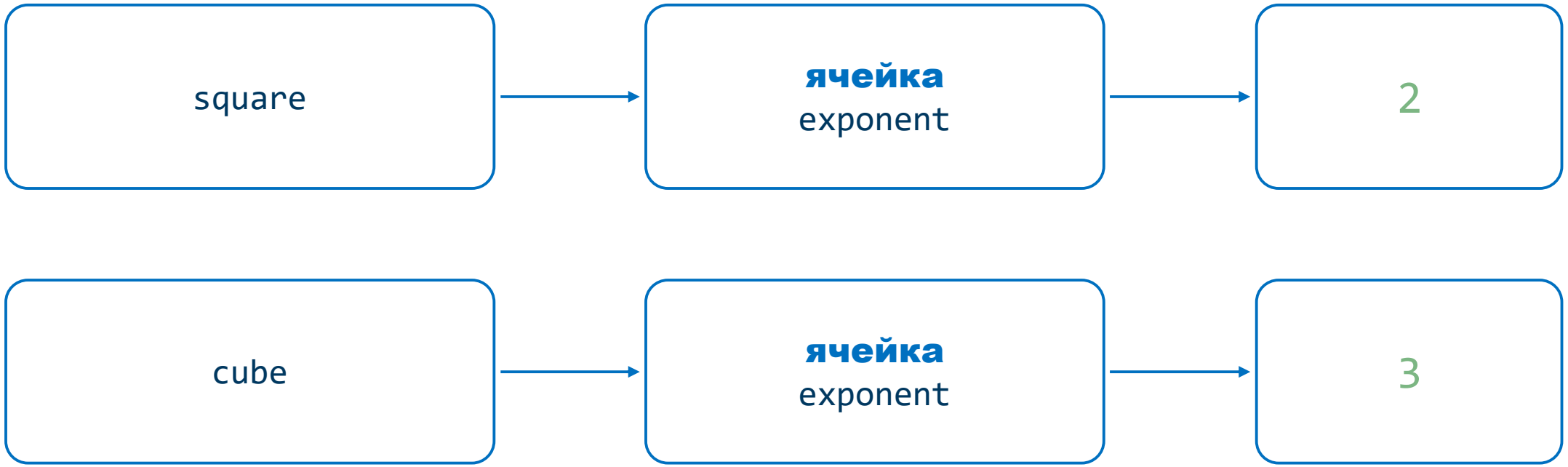
    # 3. возврат вложенной функции
    return power
```

Использование фабрики

```
square = make_power(2)
print(square(5))
# 25

cube = make_power(3)
print(cube(5))
# 125
```

Независимость замыканий



у каждого замыкания собственное состояние

Фабрики и состояния

```
from typing import Callable

def make_counter() -> Callable[[], int]:
    count = 0

    def increment() -> int:
        nonlocal count
        count += 1
        return count

    return increment
```

Пример использования

```
first_counter = make_counter()
second_counter = make_counter()

print(first_counter())   # 1
print(first_counter())   # 2
print(first_counter())   # 3
print(second_counter())  # 1
# у каждого счетчика свое состояние
```

Что дает фабрика

повторное использование логики

раздельная настройка и вызов

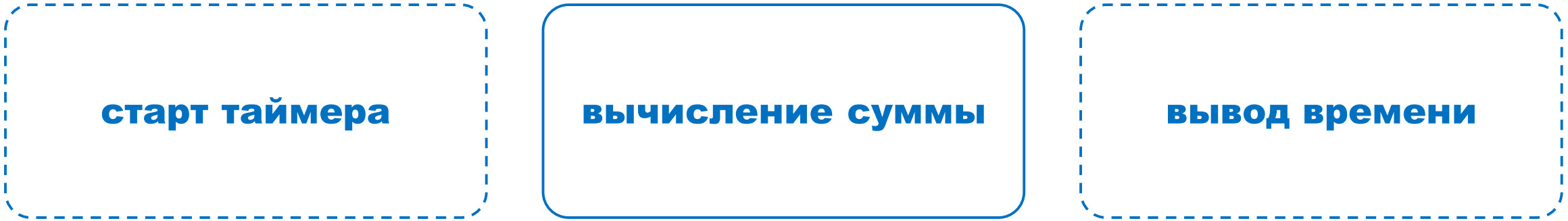
инкапсуляция состояния

независимость экземпляров



Введение в декораторы

Задача: замер времени



Замер в теле функции

```python
import time

def compute_sum(limit: int) -> int:
    start_time = time.perf_counter()
    total = 0

    for number in range(limit):
        total += number * number

    elapsed_time = time.perf_counter() - start_time
    print(f"compute_sum took {elapsed_time:.4f} s")
    return total

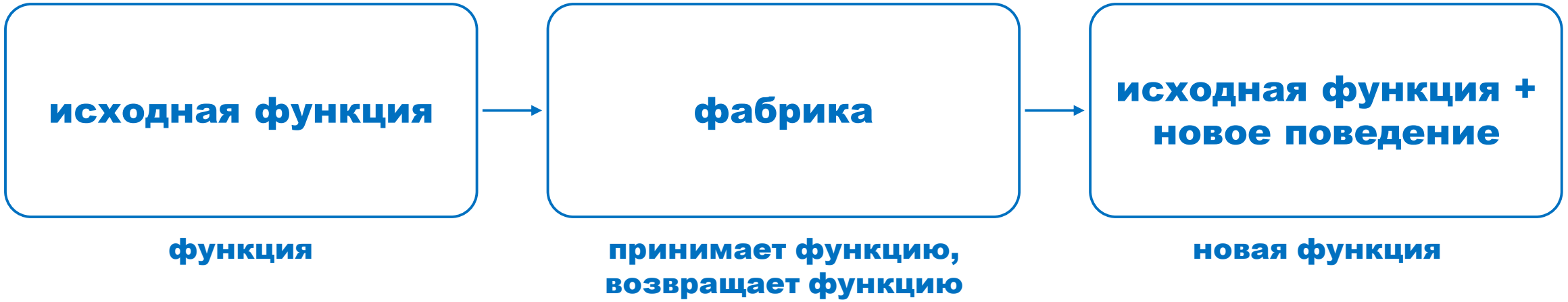
compute_sum(2_000_000)
# compute_sum took 0.1043 s
```

Недостатки решения

дублирование кода

смешение ответственностей

Использование фабрики



Реализация фабрики

```
import time
from typing import Any, Callable

def measure_time(function: Callable[..., Any]) -> Callable[..., Any]:
    def wrapper(*args: Any, **kwargs: Any) -> Any:
        start_time = time.perf_counter()
        result = function(*args, **kwargs)
        elapsed_time = time.perf_counter() - start_time
        print(f"{function.__name__} took {elapsed_time:.4f} s")
        return result

    return wrapper
```

Исходная функция

```
def compute_sum(limit: int) -> int:
    total = 0

    for number in range(limit):
        total += number * number

    return total
```

Использование фабрики

```
timed_compute_sum = measure_time(compute_sum)

print(timed_compute_sum(2_000_000))
# compute_sum took 0.1038 s
# 2666664666667000000

print(compute_sum(2_000_000))
# 2666664666667000000
```

Применение к другой функции

```
def multiply_all(numbers: list[int]) -> int:
    result = 1

    for number in numbers:
        result *= number

    return result

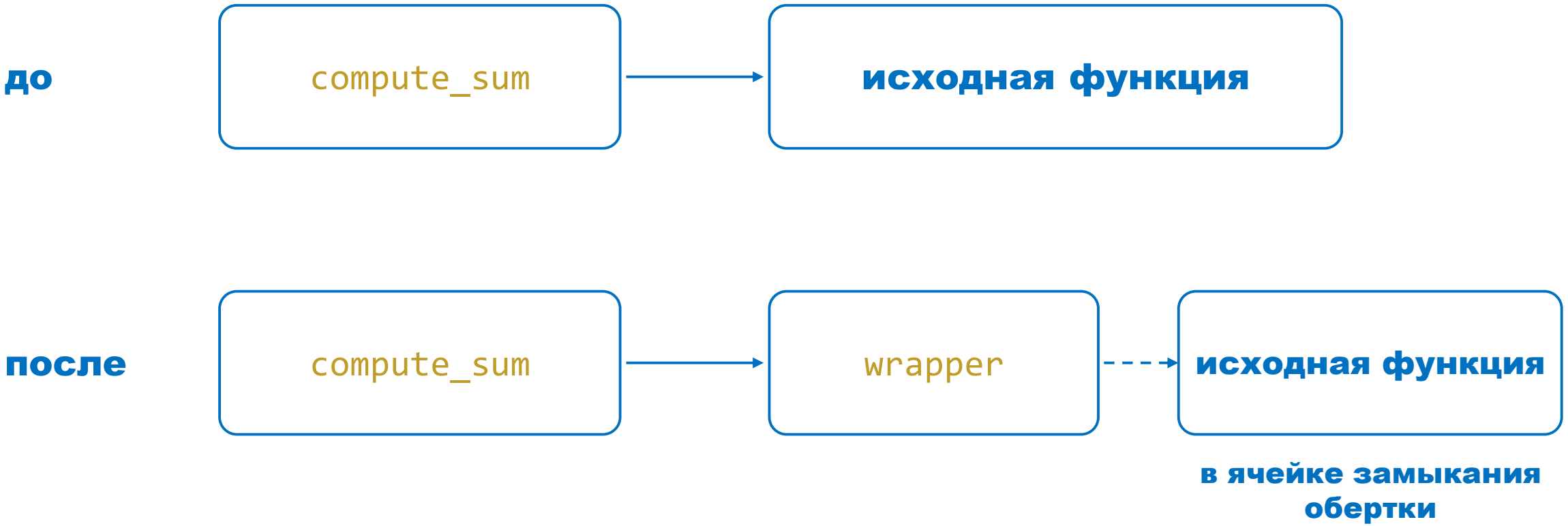
timed_multiply_all = measure_time(multiply_all)
print(timed_multiply_all([1, 2, 3, 4]))
# multiply_all took 0.0000 s
# 24
```

Перепривязка имени

```
compute_sum = measure_time(compute_sum)

print(compute_sum(2_000_000))
# compute_sum took 0.1041 s
# 2666664666667000000
```

Перепривязка имени



Синтаксис @

```
@measure_time
def compute_sum(limit: int) -> int:
    total = 0

    for number in range(limit):
        total += number * number
    return total

print(compute_sum(2_000_000))
# compute_sum took 0.1040 s
# 2666664666667000000
```

@ и вызываемый объект

```
@print
def greet() -> None:
    pass

print(greet)
# <function greet at 0x000001F2A3B4C5E0>
# None
```



Детали работы декораторов

Декоратор announce

```
from typing import Any, Callable

def announce(function: Callable[..., Any]) -> Callable[..., Any]:
    # выполняется при декорировании
    print(f"Decorating {function.__name__}")

    def wrapper(*args: Any, **kwargs: Any) -> Any:
        # выполняется при каждом вызове
        print(f"Calling {function.__name__}")
        return function(*args, **kwargs)

    return wrapper
```

Пример использования

```
@announce
def say_hello() -> None:
    print("Hello")
# Decorating say_hello

say_hello()
# Calling say_hello
# Hello
say_hello()
# Calling say_hello
# Hello
```

Декораторы и состояния

```
from typing import Any, Callable

def count_calls(function: Callable[..., Any]) -> Callable[..., Any]:
    calls_count = 0

    def wrapper(*args: Any, **kwargs: Any) -> Any:
        nonlocal calls_count
        calls_count += 1
        print(f"{function.__name__} called {calls_count} time(s)")
        return function(*args, **kwargs)

    return wrapper
```

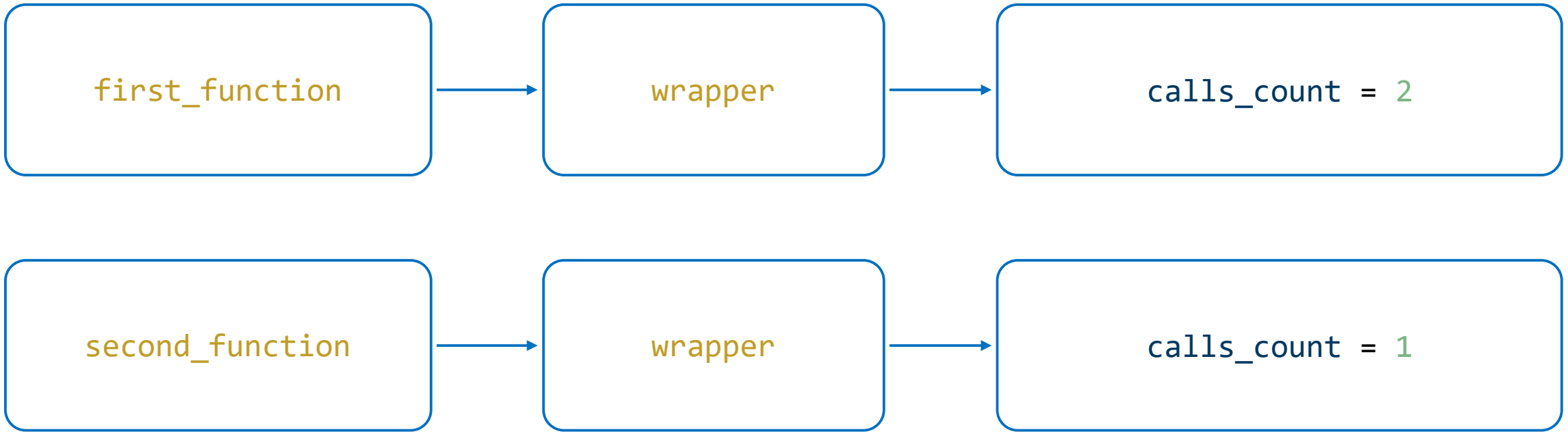
Декораторы и состояния

```
@count_calls
def first_function() -> None:
    pass

@count_calls
def second_function() -> None:
    pass

first_function()   # first_function called 1 time(s)
first_function()   # first_function called 2 time(s)
second_function()  # second_function called 1 time(s)
```

Состояние при декорировании



Время выполнения



функции еще ни разу не вызывались

Функция compute_sum

```
def compute_sum(limit: int) -> int:
    """Return the sum of squares of numbers."""
    total = 0
    for number in range(limit):
        total += number * number
    return total

print(compute_sum.__name__)
# compute_sum
print(compute_sum.__doc__)
# Return the sum of squares of numbers
print(compute_sum.__annotations__)
# {'limit': <class 'int'>, 'return': <class 'int'>}
```

Декоратор measure_time

```
import time
from typing import Any, Callable

def measure_time(function: Callable[..., Any]) -> Callable[..., Any]:
    def wrapper(*args: Any, **kwargs: Any) -> Any:
        start_time = time.perf_counter()
        result = function(*args, **kwargs)
        elapsed_time = time.perf_counter() - start_time
        print(f"{function.__name__} took {elapsed_time:.4f} s")
        return result

    return wrapper
```

Метаданные и декорирование

```
compute_sum = measure_time(compute_sum)

print(compute_sum.__name__)
# wrapper
print(compute_sum.__doc__)
# None
print(compute_sum.__annotations__)
# {'args': typing.Any, ...}
```

Подмена метаданных

	до декоратора	после декоратора
`__name__`	compute_sum	wrapper
`__doc__`	docstring	None
`__annotations__`	limit: int	args, kwargs: Any

Последствия подмены

трассировка показывает wrapper

help() описывает обертку

сигнатура (*args, **kwargs)

реестры по __name__ получают неверное имя

Ручное исправление

```
def measure_time(function: Callable[..., Any]) -> Callable[..., Any]:
    def wrapper(*args: Any, **kwargs: Any) -> Any:
        start_time = time.perf_counter()
        result = function(*args, **kwargs)
        elapsed_time = time.perf_counter() - start_time
        print(f"{function.__name__} took {elapsed_time:.4f} s")
        return result

    wrapper.__name__ = function.__name__
    wrapper.__doc__ = function.__doc__
    return wrapper
```

wraps

```
import time
from functools import wraps
from typing import Any, Callable

def measure_time(function: Callable[..., Any]) -> Callable[..., Any]:
    @wraps(function)
    def wrapper(*args: Any, **kwargs: Any) -> Any:
        ...

    return wrapper
```

Сохранение метаданных

```
@measure_time
def compute_sum(limit: int) -> int:
    """Return the sum of squares of numbers."""
    total = 0
    for number in range(limit):
        total += number * number
    return total

print(compute_sum.__name__)
# compute_sum
print(compute_sum.__doc__)
# Return the sum of squares of numbers.
print(compute_sum.__annotations__)
# {'limit': <class 'int'>, 'return': <class 'int'>}
```



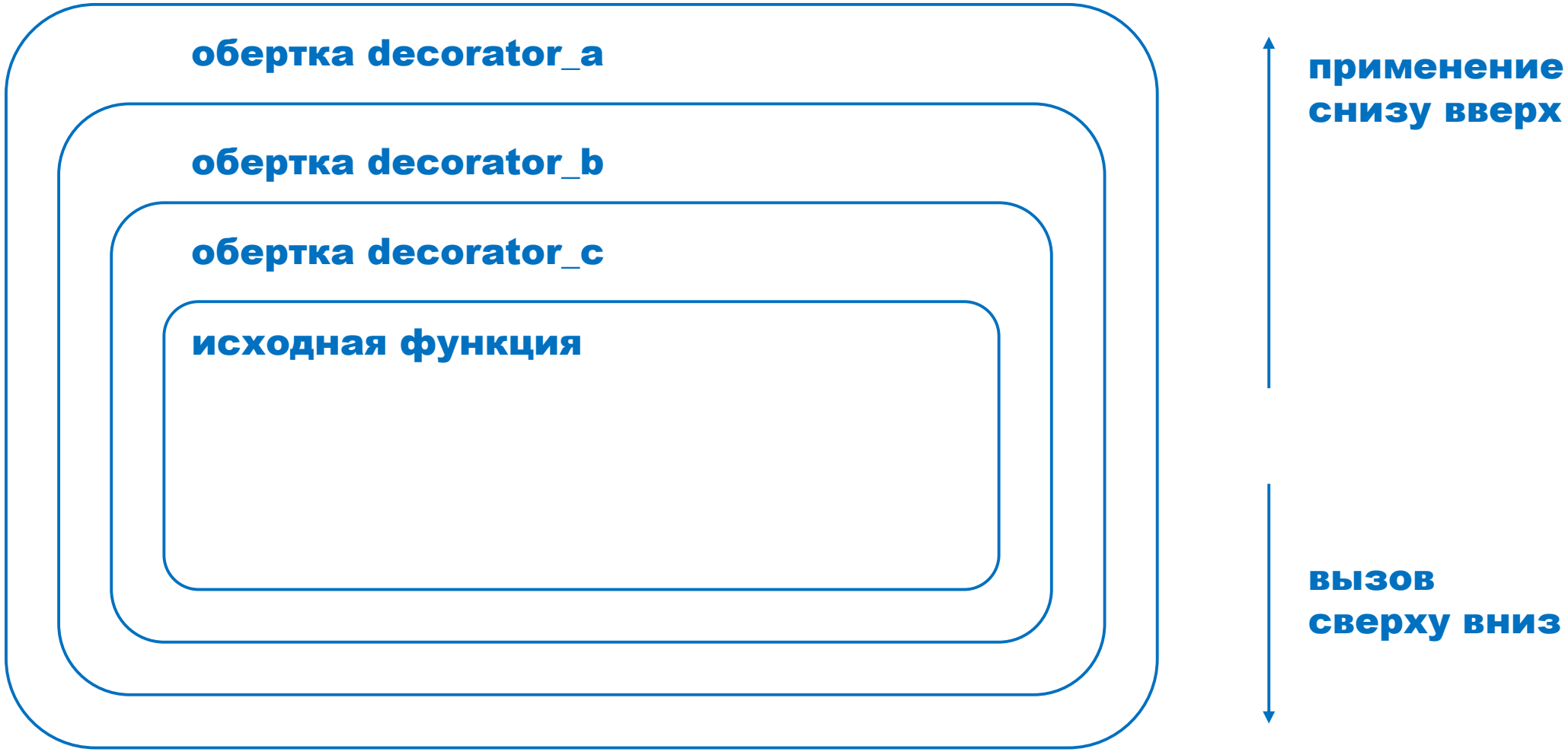
Композиция декораторов

Композиция декораторов

```
@decorator_a
@decorator_b
@decorator_c
def function() -> None:
    ...
def function() -> None:
    ...

function = decorator_a(decorator_b(decorator_c(function)))
```

Вложенность оберток



Декоратор decorator_one

```
from functools import wraps
from typing import Any, Callable

def decorator_one(function: Callable[..., Any]) -> Callable[..., Any]:
    print("Decorating with decorator_one")
    @wraps(function)
    def wrapper(*args: Any, **kwargs: Any) -> Any:
        print("decorator_one: before")
        result = function(*args, **kwargs)
        print("decorator_one: after")
        return result

    return wrapper
```

Декоратор decorator_two

```
def decorator_two(function: Callable[..., Any]) -> Callable[..., Any]:
    print("Decorating with decorator_two")
    @wraps(function)
    def wrapper(*args: Any, **kwargs: Any) -> Any:
        print("decorator_two: before")
        result = function(*args, **kwargs)
        print("decorator_two: after")
        return result

    return wrapper
```

Порядок применения и вызова

Код

```
@decorator_one
@decorator_two
def say_hello() -> None:
    print("Hello")

print()
say_hello()
```

Вывод

```
Decorating with
decorator_two
Decorating with
decorator_one

decorator_one: before
decorator_two: before
Hello
decorator_two: after
decorator_one: after
```

Влияние порядка

```
from functools import wraps
from typing import Any, Callable

def add_brackets(function: Callable[..., str]) -> Callable[..., str]:
    @wraps(function)
    def wrapper(*args: Any, **kwargs: Any) -> str:
        return f"[{function(*args, **kwargs)}]"
    return wrapper

def add_quotes(function: Callable[..., str]) -> Callable[..., str]:
    @wraps(function)
    def wrapper(*args: Any, **kwargs: Any) -> str:
        return f'"{function(*args, **kwargs)}"'
    return wrapper
```

Влияние порядка

```
@add_brackets
@add_quotes
def get_title() -> str:
    return "Python"

@add_quotes
@add_brackets
def get_same_title() -> str:
    return "Python"

print(get_title())        # ["Python"]
print(get_same_title())   # "[Python]"
```



Параметрические декораторы

Постановка задачи

```
@repeat(3)
def say_hello(name: str) -> None:
    print(f"Hello, {name}!")
```

@repeat(3): два шага



Эквивалентная запись

```
# вызов фабрики, результат - декоратор
decorator = repeat(3)

def say_hello(name: str) -> None:
    print(f"Hello, {name}!")

# применение декоратора
say_hello = decorator(say_hello)
```

Фабрика repeat

```
from functools import wraps
from typing import Any, Callable

def repeat(times: int) -> Callable[[Callable[..., Any]], Callable[..., Any]]:
    def decorator(function: Callable[..., Any]) -> Callable[..., Any]:
        @wraps(function)
        def wrapper(*args: Any, **kwargs: Any) -> Any:
            result = None
            for attempt in range(times):
                result = function(*args, **kwargs)
            return result

        return wrapper

    return decorator
```

Применение repeat

```
@repeat(3)
def say_hello(name: str) -> None:
    print(f"Hello, {name}!")

say_hello("Alice")
# Hello, Alice!
# Hello, Alice!
# Hello, Alice!
```

Разные параметры

```
@repeat(2)
def say_goodbye(name: str) -> None:
    print(f"Goodbye, {name}!")

say_goodbye("Bob")
# Goodbye, Bob!
# Goodbye, Bob!
say_hello("Bob")
# Hello, Bob!
# Hello, Bob!
# Hello, Bob!
```

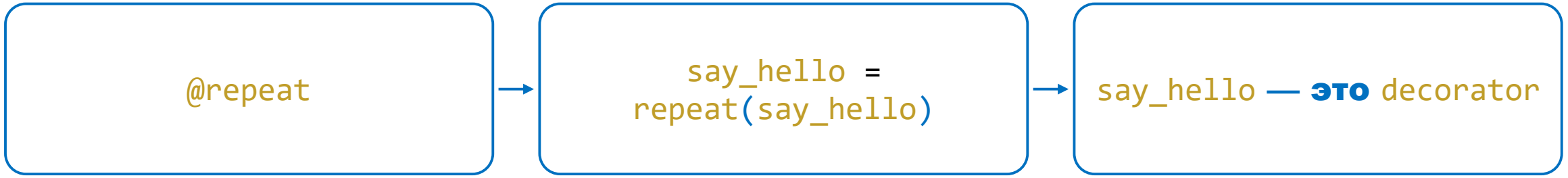
Пропущенные скобки

```
@repeat
def say_hello(name: str) -> None:
    print(f"Hello, {name}!")

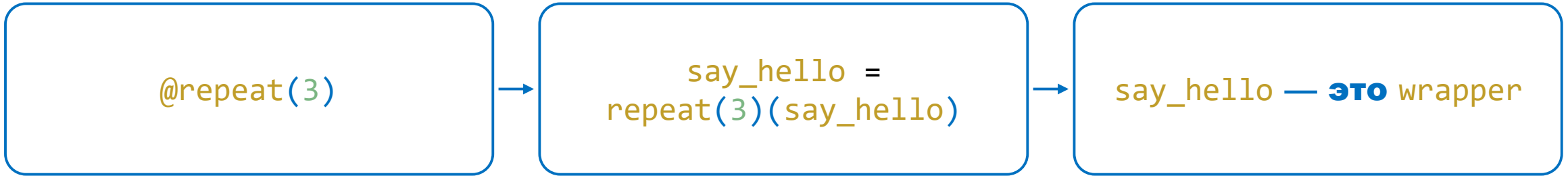
print(say_hello.__name__)
# decorator
say_hello("Alice")
# ничего не выводится
```

Скобки обязательны

без скобок



со скобками



Спасибо за внимание!